

Грамматика продвинутого лямбда-интерпретатора

Требования к языку

Входная программа состоит из произвольной последовательности именованных определений и лямбда-выражений, в том числе может быть пустой. Каждый оператор записывается на отдельной непустой строке. Определение имеет вид

```
let name = expression
```

и может использоваться в последующих выражениях как обычная переменная. Слово `let` является ключевым словом и не может быть именем переменной или определения.

Лямбда обозначается символом `\`. Должны поддерживаться абстракции с несколькими параметрами:

```
\x y z. expression
```

Такая запись является сокращением для вложенных абстракций:

$$\lambda x. \lambda y. \lambda z. \text{expression}.$$

Лексические правила

Пробельные символы между токенами игнорируются. Идентификатор начинается с буквы, далее могут идти буквы, цифры, апостроф и символ подчеркивания. Ключевое слово `let` исключается из множества идентификаторов.

```
letter      ::= "a" | ... | "z" | "A" | ... | "Z"
digit       ::= "0" | ... | "9"
identifier  ::= letter (letter | digit | "'" | "_")*
```

Факторизованная грамматика

Грамматика записана в EBNF. Оператор `+` означает «один или больше», `*` — «ноль или больше», а `?` — необязательную часть.

```
program      ::= statement*
statement    ::= (definition | expression) lineEnd
definition   ::= "let" identifier "=" expression
expression   ::= abstraction | applicationChain
abstraction  ::= "\" identifier+ "." expression
applicationChain ::= atom+ abstraction?
atom         ::= identifier | "(" expression ")"
lineEnd      ::= end of current non-empty line
```

Почему грамматика подходит для парсера

В исходной грамматике аппликация естественно записывается как

$$\text{term} ::= \text{term term},$$

что дает левую рекурсию. Для `FParsec` и рекурсивного спуска такая форма неподходящая: парсер `term` немедленно вызывает сам себя без потребления входа.

В предложенной грамматике аппликация заменена на цепочку атомов с необязательной абстракцией в конце:

$$applicationChain ::= atom^+ abstraction?.$$

Такой нетерминал парсится как `many1 atom` и `opt abstraction`. Если в цепочке один атом, результатом является сам атом. Если выражений несколько, AST строится левоассоциативно:

$$a\ b\ c \equiv ((a\ b)\ c).$$

Например, абстракция без скобок может быть последним аргументом:

$$A\ B\ \backslash x.y \equiv ((A\ B)\ (\backslash x.y)).$$

Альтернативы в правиле **expression** различаются первым токеном: абстракция начинается с `\`, а цепочка аппликации начинается с идентификатора или открывающей скобки. Поэтому правило удобно реализовать через **choice** без попыток угадывать одинаковые префиксы.

Разделение операторов по строкам нужно для однозначности. Без этого запись

```
let K = \x y.x
S K K
```

можно было бы прочесть как одно определение с телом `\x y.x S K K`. Поэтому каждая непустая строка является отдельным оператором: определением, если она начинается с ключевого слова **let**, или выражением в остальных случаях. Определения и выражения могут чередоваться.

Построение AST

Для дальнейшей реализации удобно использовать тот же базовый AST, что и в предыдущей задаче:

$$\begin{aligned} Expr &::= Var(name) \\ &| Abs(name, Expr) \\ &| App(Expr, Expr) \end{aligned}$$

Именованные определения могут храниться отдельно в таблице

$$name \mapsto Expr.$$

Перед нормализацией очередного выражения имена из таблицы можно раскрыть или учитывать их при редукции как заранее заданные переменные.

Абстракция с несколькими параметрами строится как последовательность вложенных **Abs**. Например:

```
\x y.x -> Abs(x, Abs(y, Var(x)))
```

Цепочка аппликации строится левым сворачиванием. Например:

```
S K K -> App(App(Var(S), Var(K)), Var(K))
```

Программа представляется списком операторов:

$$\begin{aligned} Statement &::= Definition(name, Expr) \\ &| Expression(Expr). \end{aligned}$$

Примеры вывода по грамматике

Пример программы с чередованием определений и выражений:

```
program
=> statement statement statement statement
=> definition lineEnd expression lineEnd definition lineEnd expression lineEnd
=> let I = \x.x
    I q
    let K = \x y.x
    K I
```

Пример аппликации, последним аргументом которой является абстракция:

```
program
=> statement
=> expression lineEnd
=> applicationChain lineEnd
=> atom atom abstraction lineEnd
=> A B \x.y
```

Пример 1

Вход:

```
let S = \x y z.x z (y z)
let K = \x y.x
```

S K K

Разбор определений:

$$\begin{aligned} S &\mapsto \text{Abs}(x, \text{Abs}(y, \text{Abs}(z, \text{App}(\text{App}(\text{Var}(x), \text{Var}(z)), \text{App}(\text{Var}(y), \text{Var}(z))))) \\ K &\mapsto \text{Abs}(x, \text{Abs}(y, \text{Var}(x))) \end{aligned}$$

Разбор итогового выражения:

S K K $\rightarrow$ App(App(Var(S), Var(K)), Var(K))

После раскрытия определений и бета-редукции ожидаемый результат:

\x.x

или любой альфа-эквивалентный терм.

Пример 2

Вход:

```
let I = \x.x
```

(\x.x) I

Разбор итогового выражения:

(\x.x) I $\rightarrow$ App(Abs(x, Var(x)), Var(I))

После раскрытия определения I и бета-редукции:

\x.x